

Anuncios

24 de septiembre de 2026



- Hoy: *Threading*.
 - Recuerden que deben revisar si se marcó bien la TUC en el excel de registro de asistencia.
 - Recuerden la ECA y la Encuesta de Mitad de Semestre
-

Control de Entrada



git add, git commit, git push

Comentarios AC4

- **Recomendación:** Lean todo el enunciado antes de empezar.
- Recuerden que esto es una **actividad individual**. **Mantengan el silencio.**
- Pueden usar `datetime.now()` para obtener el timestamp actual
- El método `contratar_cocineros()` en `Cocina`, SI se puede modificar

Cierre

Actividad 4: Threading



Que debemos identificar



Cocinero = Un thread



**Cocina = hilo principal
y coordinador**



**Pedido = Almacena
información**

Parte 1. El turno de los cocineros

Parte 1.1: Los cuatro cocineros deben trabajar de manera concurrente y cada uno atiende su propia cola.

¿Qué debemos convertir en Thread y qué método representa el turno completo?

Respuesta: Cada Cocinero es un Thread y run representa su turno. El hilo principal usa start para iniciar a todos; llamar run directamente no crea concurrencia.

Parte 1. El turno de los cocineros

Parte 1.1: Nadie puede tocar nada hasta que el restaurante abra, pero todos deben poder comenzar cuando reciba la señal.

¿Qué mecanismo permite esperar una señal compartida sin consultar una variable repetidamente?

Respuesta: Un Event de apertura. Cada cocinero hace wait y la Cocina hace set una vez que abre. La señal libera a todos los cocineros que estaban esperando.

Parte 1. El turno de los cocineros

Parte 1.1: Tras el cierre nadie inicia un pedido nuevo, pero un pedido ya iniciado debe terminarse.

¿Cómo avisamos el cierre a todos los cocineros y en qué punto del ciclo deben revisarlo?

Respuesta: Con un Event de cierre. Antes de sacar el próximo pedido de su cola, cada cocinero consulta `is_set`. Si cerró, se retira; si ya tomó un pedido, lo termina.

Parte 1. El turno de los cocineros

Parte 1.2: Solo un cocinero puede cocinar a la vez. Si un plato se quema, debe rehacerlo sin liberar la estufa.

¿Qué protege el recurso compartido?

Respuesta: Un Lock compartido. Cubre desde tomar la estufa hasta terminar de cocinar y, si corresponde, rehacer el plato. Picar y emplatar no requieren Lock porque no usan la estufa.

Parte 1. El turno de los cocineros

Parte 1.3: Las entregas deben registrarse correctamente aunque varios cocineros terminen pedidos de manera concurrente.

¿Cómo sabemos que una actualización requiere exclusión mutua y cómo calcularemos cada tiempo?

Respuesta: Si varios Threads modifican un registro común, protegemos esa actualización con un Lock. Registramos inicio al tomar el pedido y entrega al terminarlo; la diferencia entrega y inicio da su tiempo.

Parte 2. Simulación del servicio

Parte 2.2: La cocina cierra después de `duracion_servicio` segundos sin impedir que los cocineros sigan trabajando durante ese tiempo

¿Cómo programamos `cerrar_cocina` sin dejar al hilo principal detenido esperando que pase el tiempo?

Respuesta: Con un Timer. El Timer ejecuta `cerrar_cocina` después del tiempo indicado en otro Thread. El cierre activa el Event de cierre y el resto de la simulación continúa.

Parte 2. Simulación del servicio

Parte 2.3 y 2.4: El reporte solo puede construirse cuando todos los cocineros terminaron y ya no quedan entregas pendientes de registrar.

¿Qué método permite al hilo principal esperar el término de cada cocinero antes de construir el retorno?

Respuesta: join. El hilo principal hace join a cada Cocinero y recién después calcula los tiempos de los entregados y reúne los pedidos que quedaron sin atender.

Parte 3. Inicializar

Parte 3: crea una instancia de Cocina y ejecuta su método `simular_servicio` y guarda el resultado retornado.

```
from clases import Cocina
from utilidades import imprimir_reporte

def main() -> None:
    cocina = Cocina()
    resultado = cocina.simular_servicio()
```

#Completar

#Completar